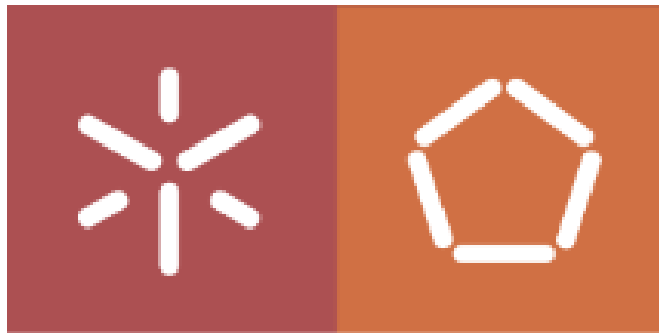


Computação Gráfica

Fase 4 - Normais e Coordenadas de Textura

Maio 2026



Universidade do Minho
Escola de Engenharia

Grupo: 8

Realizado por: Juliana Sofia Vaz da Silva, a105572

Sofia Beatriz Miranda Couto, a106925

Soraia Filipa Ribeiro Pereira, a106806

Índice

Introdução	3
1. Normais e Coordenadas de Textura	4
1.1. Nova configuração dos ficheiros .3d	4
1.1.1. Estrutura Vertex	4
1.2. Cálculo de Normais	4
1.2.1. Plane	4
1.2.2. Box	4
1.2.3. Sphere	5
1.2.4. Cone	5
1.2.5. Cylinder	5
1.2.6. Torus	6
1.2.7. Patches de Bézier	6
1.3. Coordenadas de Textura (UV)	6
1.2.1. Plane	6
1.2.2. Box	7
1.2.3. Sphere	7
1.2.4. Cone	7
1.2.5. Cylinder	7
1.2.6. Torus	8
1.2.7. Patches de Bézier	8
2. Engine	8
2.1. Iluminação	8
2.1.2. Atualização do Parser	9
2.1.3. Materiais	9
2.2. Texturas	9
2.2.1. Carregamento de Texturas	9
2.2.2. Aplicação de Texturas	9
2.2.3. Atualização do Parser e da Classe Model	10
3. Sistema Solar Animado com Texturas e Iluminação	10
4. Testes	11
Conclusão	13

Introdução

Este relatório descreve o desenvolvimento da quarta e última fase do projeto da unidade curricular de Computação Gráfica, inserida no 2º semestre do 3º ano da Licenciatura em Engenharia Informática, no ano letivo 2025/2026.

Após a implementação de curvas de *Catmull-Rom*, *patches* de *Bézier* e renderização com VBOs e IBOs na fase anterior, esta etapa teve como objetivo melhorar o realismo visual do *engine* através da introdução de iluminação, materiais, normais e texturas.

1. Normais e Coordenadas de Textura

1.1. Nova configuração dos ficheiros .3d

Na fase anterior, o ficheiro .3d já incluía uma lista de vértices únicos seguida de uma lista de índices. Na fase 4, cada linha de vértices passa também a conter informação acerca da normal no ponto e componentes UV, essenciais para a implementação da iluminação e texturas. O formato novo dos ficheiros .3d encontra-se representado na figura 1.

```
4
-0.5 0 -0.5 0 1 0 0 0
0.5 0 -0.5 0 1 0 1 0
-0.5 0 0.5 0 1 0 0 1
0.5 0 0.5 0 1 0 1 1
6
0 3 1
0 2 3
```

■ coordenadas do ponto
■ vetor normal no ponto
■ coordenadas de textura

Figura 1: Exemplo de configuração de ficheiro .3d de um plano.

A estrutura de dados **IndexedVertices**, introduzida na fase anterior, foi igualmente atualizada e passa a guardar esta nova informação:

- **position:** coordenadas do vértice (x, y, z) ;
- **normal:** vetor normal (nx, ny, nz);
- **texCoord:** coordenadas de textura (u, v).

1.1.1. Estrutura Vertex

Consequentemente, a leitura deste ficheiro no *engine* também foi alterada e o objeto **Vertex** passou a encapsular toda a informação relativa a um vértice:

- **Position:** Objeto do tipo **Point** com as coordenadas x, y e z do vértice;
- **Normal:** Objeto do tipo **Point** com as coordenadas nx, ny e nz do vetor normal;
- **texCoord:** Objeto do tipo **Point2d** com as coordenadas u e v do vetor normal.

1.2. Cálculo de Normais

O cálculo das normais foi feito diretamente nos geradores de cada primitiva, garantindo que os vetores não precisam de ser recalculados pelo *engine*.

1.2.1. Plane

O **Plane** está sempre no plano XZ, logo todos os vértices partilham a mesma normal apontada para cima:

$$n = (0, 1, 0)$$

1.2.2. Box

A **Box** é composta por seis faces planas, cada uma com a sua normal apontando para o exterior:

- Face da frente: $n = (0, 0, 1)$
- Face de trás: $n = (0, 0, -1)$
- Face de cima: $n = (0, 1, 0)$
- Face de baixo: $n = (0, -1, 0)$
- Face da direita: $n = (1, 0, 0)$
- Face da esquerda: $n = (-1, 0, 0)$

Como as normais são diferentes entre faces adjacentes, os vértices partilhados geometricamente são duplicados, ficando cada face com os seus próprios vértices.

1.2.3. Sphere

Para uma **Sphere** centrada na origem e de raio r , o vetor normal em qualquer ponto de superfície (x, y, z) é calculado através da expressão:

$$n = (x/r, y/r, z/r)$$

Para casos onde já se sabia as coordenadas do vetor normal, foi atribuído diretamente:

- Vértice e triângulos do topo: $n = (0, 1, 0)$
- Vértice do fundo: $n = (0, -1, 0)$

1.2.4. Cone

O **Cone** tem duas regiões com normais distintas: a superfície lateral e a base.

Sendo r o raio da base e h a altura, para a superfície lateral a normal é:

- $n_y = radius / \sqrt{radius^2 + height^2}$
- $n_{xz} = height / \sqrt{radius^2 + height^2}$

Para cada ponto pertencente à superfície lateral em ângulo θ , a normal é então:

- $n = (n_{xz} \times \sin(\theta), n_y, n_{xz} \times \cos(\theta))$

Para a base do cone ($y = 0$), todos os vértices têm normal $(0, -1, 0)$.

1.2.5. Cylinder

O **Cylinder** tem três regiões: a parede lateral, a base inferior e a base superior.

Para a parede lateral, a normal em ângulo azimutal θ é radial e horizontal:

$$n = (\sin(\theta), 0, \cos(\theta))$$

Para as bases inferior e superior, as normais apontam, respetivamente, para baixo e para cima:

- base inferior: $n = (0, -1, 0)$
- tampa superior: $n = (0, 1, 0)$

Por fim, é importante mencionar que os vértices laterais e das bases partilham posições geométricas mas não normais, pelo que são duplicados, tendo um a normal da parede lateral e outro a normal das bases.

1.2.6. Torus

O Torus de raio principal R e raio de tubo r é parametrizado por dois ângulos: α (em torno do eixo Y) e β (em torno do centro do tubo). A normal em qualquer ponto da superfície é a direção radial do tubo, independente de R :

$$n = (\cos(\beta) \times \cos(\alpha), \sin(\beta), \cos(\beta) \times \sin(\alpha))$$

1.2.7. Patches de Bézier

Nesta fase, as normais são calculadas analiticamente usando as derivadas parciais da superfície em (u, v) .

Dado $P(u, v)$ o ponto da superfície de Bézier, as derivadas parciais são:

$$du: \frac{\partial P}{\partial u} = \sum_i \sum_r B'_i(u) \times B_r(v) \times P_{i,r}$$

$$dv: \frac{\partial P}{\partial v} = \sum_i \sum_r B_i(u) \times B'_r(v) \times P_{i,r}$$

onde $B'_i(t)$ é a derivada do i -ésimo polinómio de *Bernstein* de grau 3, implementada na função `bernstein_derivative`:

$$B'0(t) = -3(1-t)^2$$

$$B'1(t) = 3(1-t)^2 - 6t(1-t)$$

$$B'2(t) = 6t(1-t) - 3t^2$$

$$B'3(t) = 3t^2$$

A normal é então obtida pelo produto vetorial normalizado:

$$n = \text{normalize}(dv \times du)$$

1.3. Coordenadas de Textura (UV)

As coordenadas de textura mapeiam cada vértice para um ponto de uma imagem 2D, com $u \in [0,1]$ e $v \in [0,1]$. Cada primitiva usa um mapeamento adaptado à sua geometria.

1.2.1. Plane

O Plane é dividido em $divisions \times divisions$ células, formando uma grelha. Para o vértice na linha i e coluna j da grelha:

$$u = j / divisions$$

$$v = i / divisions$$

1.2.2. Box

Cada face é mapeada de forma independente. Para as faces da frente, trás, cima e baixo, u e v seguem os eixos do sistema de coordenadas. Para as faces direita e esquerda, os eixos são trocados de modo a que a textura fique orientada coerentemente com a geometria da face.

Cada face é formada por um plano numa grelha com $divisions \times divisions$ células. Para cada linha i e coluna j :

- Faces da frente, trás, cima e baixo: $u = j / divisions;$
 $v = i / divisions;$
- Faces direita e esquerda: $u = i / divisions;$
 $v = j / divisions;$

1.2.3. Sphere

Para cada i , entre 1 e o número de *slices* e cada j entre 0 e o número de *slices*, a coordenada de textura é calculada através das expressões:

- $u = 1 - j / slices$
- $v = i / slices$

Para que a textura seja aplicada sem costuras, o vértice do topo não é partilhado entre triângulos. Em vez disso, mapeia-se cada triângulo das extremidades da seguinte forma:

- $u = 1 - ((j + 0.5) / slices)$
- $v = 0$

Já os polos vértices das extremidade tem coordenadas de textura:

- Vértice do topo: $u = 0.5$ e $v = 0$
- Vértice do fundo: $u = 0.5$ e $v = 1$

1.2.4. Cone

A superfície lateral usa mapeamento cilíndrico::

- $u = j / slices$
- $v = 1 - (i / stacks)$

A base usa mapeamento em disco centrado em (0.5, 0.5), que corresponde a:

- $u = 0.5 + 0.5 \times \sin(\theta)$
- $v = 0.5 - 0.5 \times \cos(\theta)$

O centro da base tem coordenadas de textura (0.5, 0.5). Os vértices da base são gerados independentemente das laterais para que cada região tenha o seu próprio mapeamento UV.

1.2.5. Cylinder

Para as bases do `Cylinder`, utiliza-se mapeamento em disco centrado em (0.5, 0.5):

- $u = 0.5 + 0.5 \times \sin(\theta)$
- $v = 0.5 + 0.5 \times \cos(\theta)$

A parede lateral usa mapeamento cilíndrico, totalmente isolado das bases:

- $u = i / slices$
- $v = 1.0$ no topo ou $v = 0.0$ na base

1.2.6. Torus

O torus é parametrizado por dois ângulos, α e β , mapeados diretamente para as coordenadas UV:

- $u = i / slices$ (direção do anel principal, α)
- $v = j / stacks$ (direção do tubo, β)

1.2.7. Patches de Bézier

Cada patch usa mapeamento direto dos parâmetros de tesselação, normalizados para $[0, 1]$:

- $u = row_index / tessellation_lvl$
- $v = col_index / tessellation_lvl$

Isto resulta num mapeamento planar contínuo sobre a superfície paramétrica, adequado para texturas sobre o *teapot* e outras superfícies geradas por *patches*.

2. Engine

2.1. Iluminação

Nesta fase, também foi incluído suporte a diferentes tipos de fonte de iluminação, que são definidos por diferentes componentes:

- Luz Pontual: posição (*posX*, *posY*, *posZ*)
- Luz Direcional: direção (*dirX*, *dirY*, *dirZ*)
- *Spotlight*/Luz de foco: posição (*posX*, *posY*, *posZ*), direção (*dirX*, *dirY*, *dirZ*) e ângulo de corte (*cutoff*)

Para conseguir implementar os diferentes tipos de luzes, definiu-se um enum **Type**, que define os diferentes tipos de luzes (Point, Directional e Spot), e uma classe **Light** que inclui um campo para o tipo e campos para a posição, a direção e o cutoff, usados consoante o tipo de luz.

Antes da renderização, para inicializar estas componentes, é atribuído um **lightID** a cada uma das luzes, sendo no máximo consideradas oito luzes devido ao limite do *OpenGL*. Para as renderizar, é chamada a função **drawLights()** da classe **Scene** que, para cada luz, usa as funções **glLightfv()** e **glLightf()** do *OpenGL* para definir as suas componentes. As luzes são renderizadas antes dos modelos para garantir a correta aplicação destas componentes aos modelos a renderizar.

2.1.2. Atualização do Parser

Para a configuração das luzes, o XML passou a incluir um bloco `<lights>` onde cada nó representa uma luz e é constituído por um atributo `type` e outros atributos com a ou as componentes que definem esse tipo. Foi então necessário alterar o *Parser* para fazer a leitura destes nós, instanciar os objetos de cada luz e armazená-las num vetor de luzes na classe `Scene`.

2.1.3. Materiais

Nesta fase, a classe `Model` foi estendida para suportar materiais associados a cada modelo. Para isso, foi criada a estrutura `Material`, responsável por guardar propriedades usadas pelo modelo de iluminação do *OpenGL*: *diffuse*, *ambient*, *specular*, *emissive* e *shininess*.

Estes valores são lidos a partir do bloco `<color>` definido no XML de cada modelo. A componente difusa representa a cor principal do objeto, sob iluminação direta; a ambiente define a luz mínima aplicada ao modelo; a especular controla os brilhos da superfície; a emissiva permite simular emissão de luz própria; e o *shininess* controla a concentração do brilho especular.

Durante o desenho, estas propriedades são enviadas para o *OpenGL* através de `glMaterialfv` e `glMaterialf`. Desta forma, cada modelo passa a reagir de forma diferente à iluminação da cena.

2.2. Texturas

Para além da iluminação, esta fase introduziu o suporte a texturas no *engine*. Esta funcionalidade exigiu que os modelos passassem a conter, além da sua geometria, a informação necessária para mapear imagens sobre as superfícies.

Assim, o *engine* foi adaptado para usar vértices mais completos, que contêm a posição, a normal e as coordenadas de textura, como descrito no capítulo 1.1.1. A normal é usada nos cálculos de iluminação, enquanto as coordenadas UV permitem associar cada ponto da geometria a uma posição da imagem usada como textura.

Esta alteração manteve a utilização de VBOs e IBOs introduzida na fase anterior, mas passou a enviar para a GPU uma estrutura de vértices mais completa.

2.2.1. Carregamento de Texturas

O carregamento de texturas foi implementado na classe `Model`, através do método `loadTexture()`. Para ler as imagens foi usada a biblioteca `stb_image`, que devolve os dados da imagem (largura, altura e número de canais).

Durante a leitura do XML, o *parser* verifica se o modelo contém um bloco `<texture>`. Se existir, é então lido o atributo `file`, que indica a imagem a usar como textura, ficando essa informação guardada no modelo.

No carregamento, a imagem é lida com `stbi_load` e é enviada para a GPU. Para isso, é criado um identificador com `glGenTextures`, a textura é ligada com `glBindTexture` e os seus parâmetros são configurados com `glTexParameteri`. Finalmente, os dados são enviados para o *OpenGL* com `glTexImage2D` e são gerados *mipmaps* com `glGenerateMipmap`, melhorando a qualidade visual quando o objeto é visto à distância.

2.2.2. Aplicação de Texturas

A aplicação das texturas acontece no método `drawModel()`. Antes de desenhar o modelo, o *engine* verifica se existe uma textura válida associada. Se existir, o uso de texturas é ativado com `glEnable(GL_TEXTURE_2D)` e a textura é associada ao modelo com `glBindTexture`.

As coordenadas UV, guardadas em cada **Vertex**, são passadas para o *OpenGL* através de `glTexCoordPointer` e como os dados estão todos na mesma estrutura, são também configurados os ponteiros para a posição e para a normal com o uso de `glVertexPointer` e `glNormalPointer`.

Depois dessa configuração, o modelo é desenhado com `glDrawElements` a partir de índices já existentes e no fim do desenho, a textura é desligada para evitar que afete outros modelos que não tenham textura.

2.2.3. Atualização do Parser e da Classe Model

O *parser* foi atualizado para ler o novo formato dos ficheiros `.3d` explicado no capítulo 1.1. Estes dados são armazenados num objeto **Vertex** e guardados no vetor de vértices do modelo.

Consequentemente, o vetor de vértices da classe **Model** também foi alterado e passou a armazenar objetos **Vertex**, sendo adicionados os campos para suportar texturas e materiais, como `textureID`, `textureFile`, `hasTexture`, `textureLoaded` e `material`.

O método `setupModel()` envia a nova estrutura de vértices para a GPU através do VBO, mantendo o uso do IBO para os índices. Já o método `drawModel()` passou a configurar os ponteiros para a posição, normal e coordenadas UV, além de aplicar o material e a textura antes do desenho.

3. Sistema Solar Animado com Texturas e Iluminação

Para obter a cena do sistema solar final, foram adicionadas texturas para os modelos de cada planeta e do Sol. Por uma questão de otimização, foi decidido apenas usar texturas para a lua da Terra. Para o Sol, foi adicionada uma componente emissiva, intensificada a componente difusa e a componente ambiente passou a ser nula. Também se introduziu uma *Point Light* com posição no centro do Sol para simular a luz emitida por este.

Estas alterações foram aplicadas ao *script* de geração do sistema solar, em conjunto com a opção de incluir o cometa adicionado na fase 3. Por *default*, esta opção é desligada. As texturas utilizadas encontram-se no código fonte no diretório `textures`.



Figura 2 - Sistema solar gerado com `./generate_solar_system.sh 0.9 4.5 0.3 -90`.

4. Testes

Para testar as funcionalidades implementadas nesta fase, utilizamos os ficheiros de teste disponibilizados pela equipa docente, verificando a correta aplicação da iluminação, materiais, normais e texturas nos modelos.

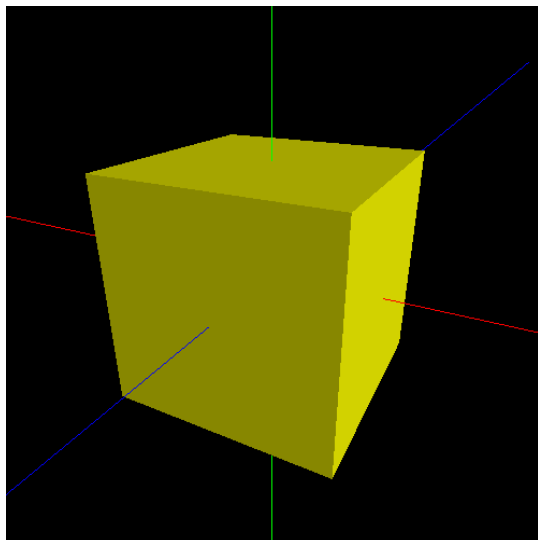


Figura 3 - Teste 1.

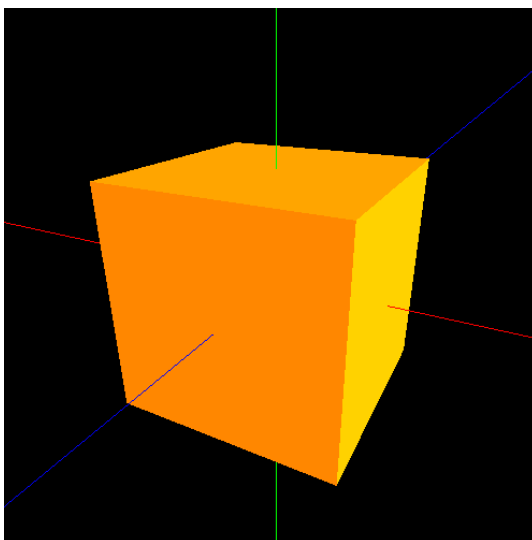


Figura 4 - Teste 2.

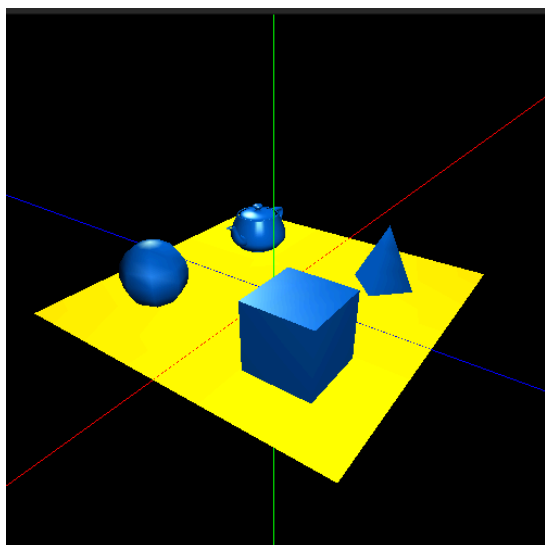


Figura 5 - Teste 3.

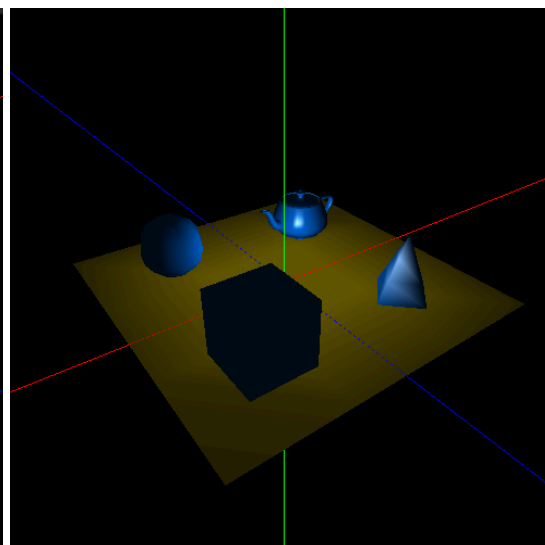


Figura 6 - Teste 4.

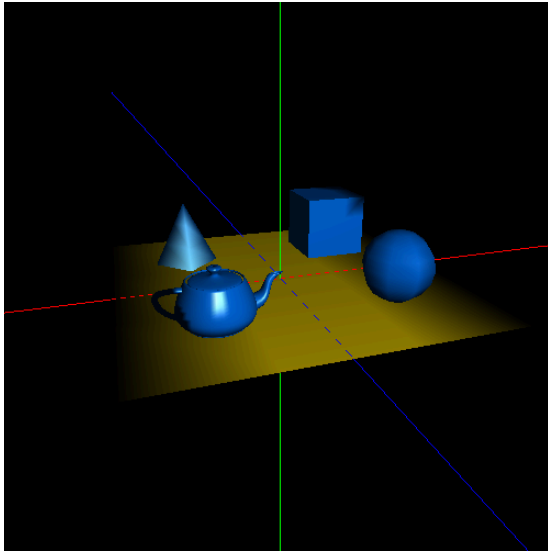


Figura 7 - Teste 5.

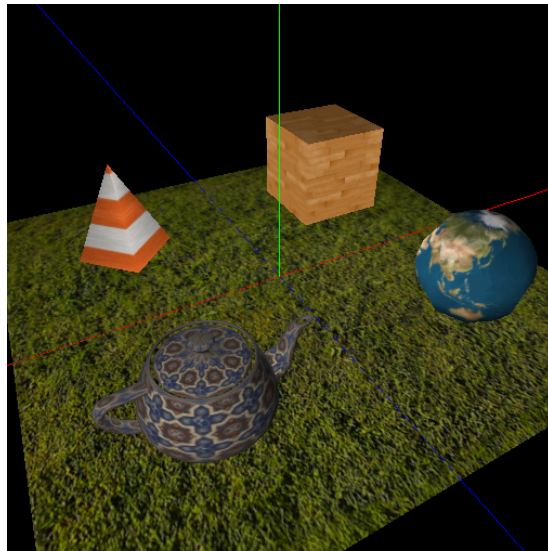


Figura 8 - Teste 6.

Conclusão

Consideramos que a quarta e última fase do projeto foi implementada com sucesso, concluindo a evolução do *engine* e do *generator* ao longo das várias etapas do trabalho.

Na primeira fase, foram desenvolvidas as bases do projeto com a criação do *generator* para produzir primitivas geométricas e do *engine* para ler os ficheiros .3d e apresentar os modelos na cena. Na segunda fase, foram introduzidas transformações geométricas e uma estrutura hierárquica de grupos, permitindo construir cenas mais organizadas e complexas. Posteriormente, na terceira fase, foram implementadas animações dependentes de tempo, curvas de *Catmull-Rom*, *patches* de *Bézier* e a utilização de VBOs e IBOs, tornando o sistema mais eficiente e dinâmico.

Nesta última fase, o foco passou a ser o realismo visual. O *generator* foi atualizado para produzir modelos com normais e coordenadas de textura, enquanto o *engine* passou a suportar iluminação, materiais e texturas. Estas alterações, permitiram que os modelos deixassem de ser representados apenas pela sua geometria, passando também a reagir às luzes na cena e a apresentar detalhes visuais através de imagens aplicadas às suas superfícies.

Ao longo do projeto, surgiram algumas dificuldades. Na parte das texturas, foi necessário garantir a continuidade das coordenadas UV nas zonas de junção, evitando *texture seams*. Além disso, a definição correta das normais e a sua interação com luzes e materiais exigiu vários ajustes para que o modelo de iluminação produzisse o resultado esperado. Tentamos ainda implementar *frustum culling*, uma técnica de otimização que evita desenhar objetos fora do volume de visão de câmara, mas esta funcionalidade acabou por não ficar concluída devido à complexidade da sua integração com a hierarquia e transformações da cena.

No geral, o trabalho permitiu consolidar os conceitos abordados nas aulas e compreender melhor o processo de construção de uma cena tridimensional, desde a modelação geométrica até à animação, iluminação e aplicação de texturas.